



Revisão Acelerada de Programação - RAP RAP-2026, Marinha do Brasil

Centro de Instrução Almirante Alexandrino

Departamento de Aperfeiçoamento Avançado para Oficiais

Curso de Aperfeiçoamento Avançado em Guerra Acústica (C-ApA-Of-GA)

Professor: João Victor da Fonseca Pinto

Descrição do Documento: **Avaliação Escrita 01**

Valor Total: **100,0 Pontos** — Nota Obtida: _____

Aluno: _____ NIP: _____

Avaliação Escrita 01 — Revisão Acelerada de Programação (RAP)

Orientações e Instruções para a Realização da Prova:

- Esta avaliação é composta por **15 questões** e totaliza **100,0 pontos**.
- **Estrutura da Prova:**
 - **Parte I (Questões 1 a 7):** 7 Questões de Múltipla Escolha (Total: **30,0 pontos**).
 - **Parte II (Questões 8 a 15):** 8 Questões Discursivas e de Implementação em Python (Total: **70,0 pontos**).
- **CrITÉrios de Correção e Resolução:**
 - Nas questões de múltipla escolha, assinale com clareza a alternativa correta no espaço indicado.
 - Nas questões discursivas, escreva o código Python de forma legível respeitando a indentação obrigatória, nomes de funções e parâmetros indicados nos protótipos.
 - É proibido o uso de módulos externos não autorizados (utilize apenas funções nativas do Python).

Parte I: Questões de Múltipla Escolha (Q1 a Q7 — 30,0 Pontos)

Q1. [Múltipla Escolha - 4,0 pontos] Considere a seguinte expressão aritmética executada em Python para calcular o valor total de uma compra com desconto:

```
total = 10 + 2 * 3 ** 2 - 8 // 4 + 7 % 3
```

Qual será o valor final atribuído à variável `total`?

- **Dica:** A ordem de precedência dos operadores aritméticos em Python é: 1º Parênteses `()`, 2º Exponenciação `**`, 3º Multiplicação `*`, Divisão `/`, Divisão Inteira `//` e Módulo `%` (avaliados da esquerda para a direita), 4º Adição `+` e Subtração `-` (avaliados da esquerda para a direita). O operador de divisão inteira `//` retorna apenas a parte inteira da divisão (ex: `8 // 4` resulta em `2`), enquanto o operador módulo `%` calcula o resto da divisão inteira (ex: `7 % 3` resulta em `1`).

- (A) 25
- (B) 27
- (C) 28
- (D) 30
- (E) 19

RESPOSTA DA QUESTÃO:

Assinale a alternativa: ☐ A ☐ B ☐ C ☐ D ☐ E

RASCUNHO / MEMÓRIA DE CÁLCULO:

1

2

Q2. [Múltipla Escolha - 4,0 pontos] Em Python, a avaliação de expressões lógicas com os operadores `and` e `or` segue a regra do curto-circuito (*short-circuit evaluation*). Considere as variáveis:

```
x = 0
y = 10
z = 20
```

Analise a expressão lógica:

```
resultado = (x != 0 and (y / x) > 2) or (z > y and y > 0)
```

Qual é o valor final atribuído a `resultado` e qual comportamento ocorre durante a execução?

- (A) Ocorre um erro de divisão por zero (`ZeroDivisionError`) ao tentar avaliar `y / x`.
- (B) O valor é `True`, pois a primeira parte da conjunção `x != 0` é `False`, evitando a divisão por zero, e a segunda parte do `or` é `True`.
- (C) O valor é `False`, pois a primeira expressão é falsa e invalida todo o teste lógico.
- (D) O valor é `None`, pois expressões com curto-circuito em Python não retornam valores booleanos puros.
- (E) O programa trava em um laço infinito aguardando a definição do operador de curto-circuito.

RESPOSTA DA QUESTÃO:

Assinale a alternativa: ☐ A ☐ B ☐ C ☐ D ☐ E

RASCUNHO / MEMÓRIA DE CÁLCULO:

1

2

Q3. [Múltipla Escolha - 4,0 pontos] Dada a string `s = "PROGRAMACAO"`, qual será o resultado da operação de fatiamento `s[1:8:2]`?

- (A) "RPAM"
- (B) "ROAM"
- (C) "PGMA"
- (D) "RMA"
- (E) "ROMA"

RESPOSTA DA QUESTÃO:

Assinale a alternativa: ☐ A ☐ B ☐ C ☐ D ☐ E

RASCUNHO / MEMÓRIA DE CÁLCULO:

1

2

Q4. [Múltipla Escolha - 4,0 pontos] Em Python, a atribuição direta entre variáveis que apontam para listas cria uma referência (*alias*), enquanto o método `.copy()` ou o fatiamento `[:]` cria uma cópia rasa (*shallow copy*). Analise o código abaixo:

```
lista_a = [10, 20, 30]
lista_b = lista_a
lista_c = lista_a.copy()
```

```
lista_b.append(40)
lista_c.append(50)
```

Quais serão os valores finais de `lista_a`, `lista_b` e `lista_c`, respectivamente?

- (A) [10, 20, 30], [10, 20, 30, 40] e [10, 20, 30, 50]
- (B) [10, 20, 30, 40, 50], [10, 20, 30, 40, 50] e [10, 20, 30, 50]
- (C) [10, 20, 30], [10, 20, 30, 40] e [10, 20, 30]
- (D) [10, 20, 30, 40], [10, 20, 30, 40] e [10, 20, 30, 50]
- (E) [10, 20, 30, 50], [10, 20, 30, 40] e [10, 20, 30, 50]

RESPOSTA DA QUESTÃO:

Assinale a alternativa: ☐ A ☐ B ☐ C ☐ D ☐ E

RASCUNHO / MEMÓRIA DE CÁLCULO:

1

2

Q5. [Múltipla Escolha - 4,0 pontos] Sobre a eficiência temporal da busca de elementos com o operador `in` e as propriedades estruturais de **Listas**, **Dicionários** e **Conjuntos** (*Sets*) em Python, assinale a alternativa **CORRETA**:

- (A) A verificação `item in lista` tem complexidade média $O(1)$ (tempo constante), pois listas utilizam tabelas de dispersão (*hash tables*) internamente.

- (B) Qualquer tipo de dado do Python pode ser utilizado como chave de um dicionário, inclusive listas e outros dicionários mutáveis.
- (C) A verificação `chave in dicionario` e `elemento in conjunto` tem complexidade média $O(1)$, pois ambos utilizam tabelas hash indexadas pelo valor de hash (`hash()`) dos seus elementos imutáveis.
- (D) Conjuntos (*sets*) mantêm estritamente a ordem de inserção dos elementos e permitem o armazenamento de itens duplicados.
- (E) Dicionários em Python não permitem a alteração dinâmica de seus valores após a instânciação inicial.

RESPOSTA DA QUESTÃO:

Assinale a alternativa: ☐ A ☐ B ☐ C ☐ D ☐ E

RASCUNHO / MEMÓRIA DE CÁLCULO:

1

2

Q6. [Múltipla Escolha - 5,0 pontos] Analise a execução do seguinte trecho de código em Python envolvendo escopo de variáveis e funções:

```
taxa = 1.10
```

```
def atualizar_precos(valores: list[float]) -> float:
    global taxa
    taxa = 1.20
    soma = 0.0
    for i in range(len(valores)):
        valores[i] = valores[i] * taxa
        soma += valores[i]
    return soma
```

```
precos = [10.0, 20.0]
```

```
total_calculado = atualizar_precos(precos)
```

Após a execução completa, quais serão os valores de `precos`, `total_calculado` e da variável global `taxa`?

- (A) `precos = [10.0, 20.0]`, `total_calculado = 33.0` e `taxa = 1.10`
- (B) `precos = [12.0, 24.0]`, `total_calculado = 36.0` e `taxa = 1.20`
- (C) `precos = [11.0, 22.0]`, `total_calculado = 33.0` e `taxa = 1.20`
- (D) `precos = [12.0, 24.0]`, `total_calculado = 36.0` e `taxa = 1.10`
- (E) Ocorre um erro de compilação (`UnboundLocalError`) devido ao uso de `global taxa`.

RESPOSTA DA QUESTÃO:

Assinale a alternativa: ☐ A ☐ B ☐ C ☐ D ☐ E

RASCUNHO / MEMÓRIA DE CÁLCULO:

1

2

Q7. [Múltipla Escolha - 5,0 pontos] Na Programação Orientada a Objetos em Python, considere o seguinte modelo de classes:

```
class Sensor:
    def __init__(self, identificador: str):
        self.id = identificador

    def obter_status(self) -> str:
        return f"Sensor {self.id}: ATIVO"

class SensorSonar(Sensor):
    def __init__(self, identificador: str, profundidade_max: float):
        super().__init__(identificador)
        self.profundidade = profundidade_max

    def obter_status(self) -> str:
        msg_base = super().obter_status()
        return f"{msg_base} (Prof: {self.profundidade}m)"
```

Ao instanciar `s = SensorSonar("SN-01", 300.0)` e executar `s.obter_status()`, qual será o retorno produzido?

- (A) "Sensor SN-01: ATIVO"
- (B) "(Prof: 300.0m)"
- (C) Ocorre um erro de execução (`AttributeError`), pois `super()` não pode acessar métodos com o mesmo nome.
- (D) "Sensor SN-01: ATIVO (Prof: 300.0m)"
- (E) "Sensor: None (Prof: 300.0m)"

RESPOSTA DA QUESTÃO:

Assinale a alternativa: ☐ A ☐ B ☐ C ☐ D ☐ E

RASCUNHO / MEMÓRIA DE CÁLCULO:

1

2

Parte II: Questões Discursivas e de Programação (Q8 a Q15 — 70,0 Pontos)

Q8. [Discursiva - 8,0 pontos] Um caixa eletrônico de autoatendimento precisa dispensar o menor número de cédulas possível para um determinado valor de saque inteiro (múltiplo de 10). O caixa dispõe de cédulas de R\$ 50, R\$ 20 e R\$ 10. Sem utilizar laços ou condicionais, escreva uma função chamada `sacar_cedulas` que receba o valor total do saque e retorne uma tupla com a quantidade de cédulas de 50, 20 e 10, nesta ordem.

- **Dica:** O operador de divisão inteira `//` obtém a quantidade inteira de vezes que um número cabe no outro (ex.: `180 // 50` resulta em `3`), enquanto o operador módulo `%` obtém o resto da divisão (ex.: `180 % 50` resulta em `30`).

- **Protótipo:**

```
from typing import Tuple
def sacar_cedulas(
    valor_reais: int
) -> Tuple[int, int, int]:
```

- **Exemplo de Entrada:** `180` — **Saída Esperada:** `(3, 1, 1)`

ESPAÇO PARA RESOLUÇÃO / CÓDIGO PYTHON:

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

FOLHA DE CONTINUACAO / RASCUNHO – QUESTAO Q8

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

27

28

29

30

Q9. [Discursiva - 8,0 pontos] Escreva uma função chamada `validar_cadastro_usuario` que valide os dados de cadastro de um novo cliente em um sistema bancário digital. Os critérios obrigatórios são:

- 1. O nome não pode ser vazio e deve ter pelo menos 3 caracteres.
- 2. A idade deve ser maior ou igual a 18 anos.
- 3. O e-mail deve conter obrigatoriamente o caractere "@" e pelo menos um caractere ponto ".".
- 4. A renda mensal informada deve ser estritamente positiva (> 0).

A função deve retornar uma tupla (`valido: bool, motivo: str`). Se todos os critérios forem atendidos, retorne `(True, "CADASTRO_APROVADO")`. Caso algum critério seja violado, retorne `False` acompanhado do primeiro motivo de recusa correspondente: `"NOME_INVALIDO"`, `"MENOR_DE_IDADE"`, `"EMAIL_INVALIDO"` ou `"RENDA_INVALIDA"`.

- **Protótipo:**

```
from typing import Tuple
def validar_cadastro_usuario(
    nome: str,
    idade: int,
    email: str,
    renda: float
) -> Tuple[bool, str]:
```

- **Exemplo de Entrada:** `("Carlos Silva", 25, "carlos@banco.com", 3500.0)`
- **Saída Esperada:** `(True, "CADASTRO_APROVADO")`

ESPAÇO PARA RESOLUÇÃO / CÓDIGO PYTHON:

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

FOLHA DE CONTINUACAO / RASCUNHO – QUESTAO Q9

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

27

28

29

30

Q10. [Discursiva - 8,0 pontos] A Cifra de César é uma técnica clássica de criptografia por substituição na qual cada letra de uma mensagem em maiúsculas é deslocada um número fixo de posições no alfabeto (considerando o alfabeto de 'A' a 'Z' de 26 letras de forma circular). Caracteres que não sejam letras maiúsculas (como espaços, números e pontuações) devem ser mantidos inalterados. Escreva uma função chamada `cifrar_texto` que receba a mensagem e o deslocamento inteiro, retornando o texto cifrado.

- **Dica:** A função `ord('A')` retorna 65 e `chr(65)` retorna 'A'. Para calcular a nova letra com deslocamento circular, você pode usar a fórmula: `chr((ord(c) - ord('A')) + deslocamento) % 26 + ord('A'))`.

- **Protótipo:**

```
def cifrar_texto(  
    mensagem: str,  
    deslocamento: int  
    ) -> str:
```

- **Exemplo de Entrada:** ("PYTHON 2026", 3) — **Saída Esperada:** "SBWKRQ 2026"

ESPAÇO PARA RESOLUÇÃO / CÓDIGO PYTHON:

1	
2	
3	
4	
5	
6	
7	
8	
9	
10	
11	
12	
13	
14	
15	
16	
17	
18	

FOLHA DE CONTINUACAO / RASCUNHO – QUESTAO Q10

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

27

28

29

30

Q11. [Discursiva - 8,0 pontos] Um sistema de e-commerce recebe uma lista de dicionários representando pedidos concluídos. Cada pedido contém: **"categoria"** (string) e **"valor"** (float). Escreva uma função chamada **consolidar_vendas_categoria** que processe essa lista e retorne um dicionário onde cada chave é o nome da categoria e o valor é outro dicionário contendo o total faturado (**"total"**), a quantidade de itens vendidos (**"quantidade"**) e o valor médio por item (**"media"**), com a média arredondada para duas casas decimais.

- **Protótipo:**

```
from typing import List, Dict, Any
def consolidar_vendas_categoria(
    vendas: List[Dict[str, Any]]
) -> Dict[str, Dict[str, Any]]:
```

- **Exemplo de Entrada:**

```
vendas = [{"categoria": "Eletronicos", "valor": 1200.0}, {"categoria":
"Livros", "valor": 50.0}]
```

- **Saída Esperada:**

```
{"Eletronicos": {"total": 1200.0, "quantidade": 1, "media": 1200.0},
"Livros": {...}}
```

ESPAÇO PARA RESOLUÇÃO / CÓDIGO PYTHON:

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

FOLHA DE CONTINUACAO / RASCUNHO – QUESTAO Q11

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

27

28

29

30

Q12. [Discursiva - 9,0 pontos] Um número natural maior do que 1 é dito **primo** se for divisível apenas por 1 e por ele mesmo. Escreva uma função em Python chamada `filtrar_primos_intervalo` que receba dois números inteiros positivos `inicio` e `fim` (com $\text{inicio} \leq \text{fim}$) e, utilizando laços de repetição (`for` ou `while`) e lógica booleana, retorne uma **lista** contendo todos os números primos existentes no intervalo fechado $[\text{inicio}, \text{fim}]$ em ordem crescente. Caso não haja nenhum primo no intervalo, retorne uma lista vazia `[]`.

- **Dica:** Para verificar se um número n é primo, basta testar se algum inteiro d no intervalo $2 \leq d \leq \sqrt{n}$ divide n de forma exata ($n \pmod{d} == 0$). Lembre-se de que números ≤ 1 não são primos.

- **Protótipo:**

```
from typing import List
def filtrar_primos_intervalo(
    inicio: int,
    fim: int
) -> List[int]:
```

- **Exemplo de Entrada:** `(10, 30)` — **Saída Esperada:** `[11, 13, 17, 19, 23, 29]`

ESPAÇO PARA RESOLUÇÃO / CÓDIGO PYTHON:

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

FOLHA DE CONTINUACAO / RASCUNHO – QUESTAO Q12

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

27

28

29

30

Q13. [Discursiva - 9,0 pontos] Um servidor de microsserviços gera linhas de log de acesso no formato: "METODO /endpoint TEMPO_MS STATUS". Por exemplo: "GET /api/v1/usuarios 45.2 200" ou "POST /api/v1/login 120.0 200".

Escreva uma função chamada `analisar_logs_servidor` que receba uma lista de strings de log e retorne uma tupla contendo:

- 1. Um dicionário mapeando cada método HTTP ("GET", "POST", "PUT", etc.) à quantidade total de requisições registradas para esse método.
- 2. O tempo médio de resposta de todas as requisições (em milissegundos), arredondado para duas casas decimais. Caso a lista esteja vazia, o tempo médio deve ser 0.0.

• **Protótipo:**

```
from typing import List, Dict, Tuple
def analisar_logs_servidor(
    linhas_log: List[str]
) -> Tuple[Dict[str, int], float]:
```

- **Exemplo de Entrada:** ["GET /home 30.0 200", "POST /login 90.0 200", "GET /dados 60.0 200"]
- **Saída Esperada:** ("GET": 2, "POST": 1, 60.0)

ESPAÇO PARA RESOLUÇÃO / CÓDIGO PYTHON:

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

FOLHA DE CONTINUACAO / RASCUNHO – QUESTAO Q13

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

27

28

29

30

Q14. [Discursiva - 10,0 pontos] Em um sistema de pagamentos, transações são representadas por dicionários com os campos: "id" (int), "tipo" ("DEPOSITO" ou "SAQUE") e "valor" (float). Escreva a função `processar_transacoes_bancarias(transacoes, saldo_inicial)` aplicando tratamento explícito de exceções (`try / except`):

- Se faltar alguma chave obrigatória ("id", "tipo" ou "valor"), capture o `KeyError` e adicione "Erro na transacao: Chave ausente" à lista de erros.
- Se o "valor" for ≤ 0 , lance e capture `ValueError` adicionando "Transacao <id>: Valor invalido" à lista de erros.
- Se for um "SAQUE" com saldo insuficiente, lance e capture `ValueError` com a mensagem "Transacao <id>: Saldo insuficiente".
- Se for válida: atualize o saldo e adicione o "id" à lista de sucessos.

Retorne a tupla: `(saldo_final: float, ids_sucesso: list[int], erros: list[str])`.

- **Protótipo:**

```
from typing import List, Dict, Any, Tuple
def processar_transacoes_bancarias(
    transacoes: List[Dict[str, Any]],
    saldo_inicial: float
) -> Tuple[float, List[int], List[str]]:
```

- **Exemplo de Entrada:**

```
trans = [{"id": 1, "tipo": "DEPOSITO", "valor": 500.0},
         {"id": 2, "tipo": "SAQUE", "valor": 700.0}]
processar_transacoes_bancarias(trans, saldo_inicial=100.0)
```

- **Saída Esperada:**

```
(600.0, [1], ["Transacao 2: Saldo insuficiente"])
```

ESPAÇO PARA RESOLUÇÃO / CÓDIGO PYTHON:

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

FOLHA DE CONTINUACAO / RASCUNHO – QUESTAO Q14

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

27

28

29

30

Q15. [Discursiva - 10,0 pontos] Modele um sistema orientado a objetos para gerenciar contas de clientes em um banco digital. Implemente a seguinte hierarquia de classes:

- **1. Classe ContaBancaria:**

- Construtor:
`__init__(self, titular: str, numero_conta: int, saldo=0.0)`
- Método `depositar(self, valor: float) -> bool`: se `valor > 0`, soma ao saldo e retorna `True`; caso contrário, retorna `False`.
- Método `sacar(self, valor: float) -> bool`: se `0 < valor <= self.saldo`, desconta do saldo e retorna `True`; caso contrário, retorna `False`.
- Método especial `__str__(self) -> str`: retorna string no formato:
`"Conta <num> - Titular: <nome> - Saldo: R$ <saldo>"`

- **2. Classe ContaEspecial (herda de ContaBancaria):**

- Construtor:
`__init__(self, titular: str, num: int, saldo=0.0, limite=500.0)`
Chama o construtor da superclasse via `super().__init__` e inicializa `limite`.
- Sobrescrita de `sacar(self, valor: float) -> bool`: permite o saque se `0 < valor <= (self.saldo + self.limite)`. Desconta do saldo e retorna `True`; caso contrário, retorna `False`.
- Sobrescrita de `__str__(self) -> str`: chama `super().__str__()` e concatena o limite:
`"<msg_base> (Limite: R$ <limite>)"`.

- **Exemplo de Uso:**

```
c = ContaEspecial("Ana Costa", 1001, saldo_inicial=200.0,
                  limite_cheque_especial=300.0)
c.sacar(450.0) # Retorna True e o saldo fica -250.0
print(str(c)) # 'Conta 1001 - Titular: Ana Costa (Limite: R$ 300.00)'
```

ESPAÇO PARA RESOLUÇÃO / CÓDIGO PYTHON:

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

FOLHA DE CONTINUACAO / RASCUNHO – QUESTAO Q15

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

27

28

29

30